



Security Assessment

T-2 | DAO Dapp

Verified on 07/26/2026

SUMMARY

Project

T-2 | DAO

CHAIN

Ethereum

METHODOLOGY

Manual & Automatic Analysis

FILES

Repository

DELIVERY

07/26/2026

TYPE

Dapp Audit



6

0

3

0

3

0

1

Total Findings

Critical

Major

Medium

Minor

Informational

Resolved

 0 Critical

An exposure that can affect the dapp's functions in several events that can risk and disrupt the code

 3 Major

An opening & exposure to manipulate the code in an unwanted manner

 0 Medium

An opening that could affect the outcome in executing the code in a specific situation

 3 Minor

An opening but doesn't have an impact on the functionality of the code

 0 Informational

An opening that consists information but will not risk or affect the code

 1 Resolved

ContractWolf's findings has been acknowledged & resolved by the project

STATUS
 **AUDIT PASSED**

TABLE OF CONTENTS | T-2 | DAO

| Summary

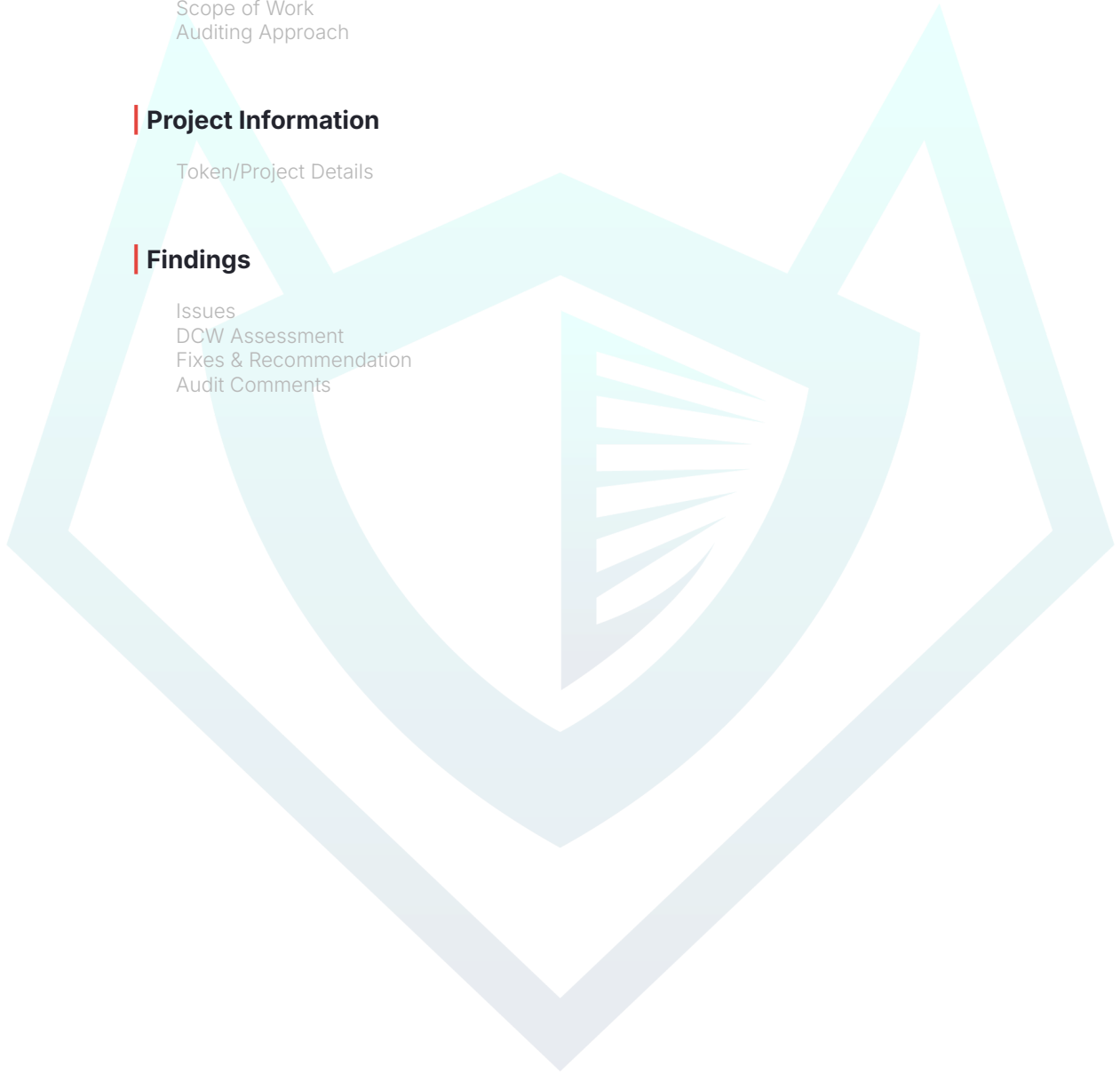
Project Summary
Findings Summary
Disclaimer
Scope of Work
Auditing Approach

| Project Information

Token/Project Details

| Findings

Issues
DCW Assessment
Fixes & Recommendation
Audit Comments



DISCLAIMER | T-2 | DAO

ContractWolf audits and reports should not be considered as a form of project's "Advertisement" and does not cover any interaction and assessment from "Project Code" to "External Code"

ContractWolf does not provide any warranty on its released report and should not be used as a decision to invest into audited projects.

ContractWolf provides a transparent report to all its "Clients" and to its "Clients Participants" and will not claim any guarantee of bug-free code within its **DAPP**.

ContractWolf's presence is to analyze, audit and assess the Client's Dapp to find any underlying risk and to eliminate any logic and flow errors within its code.

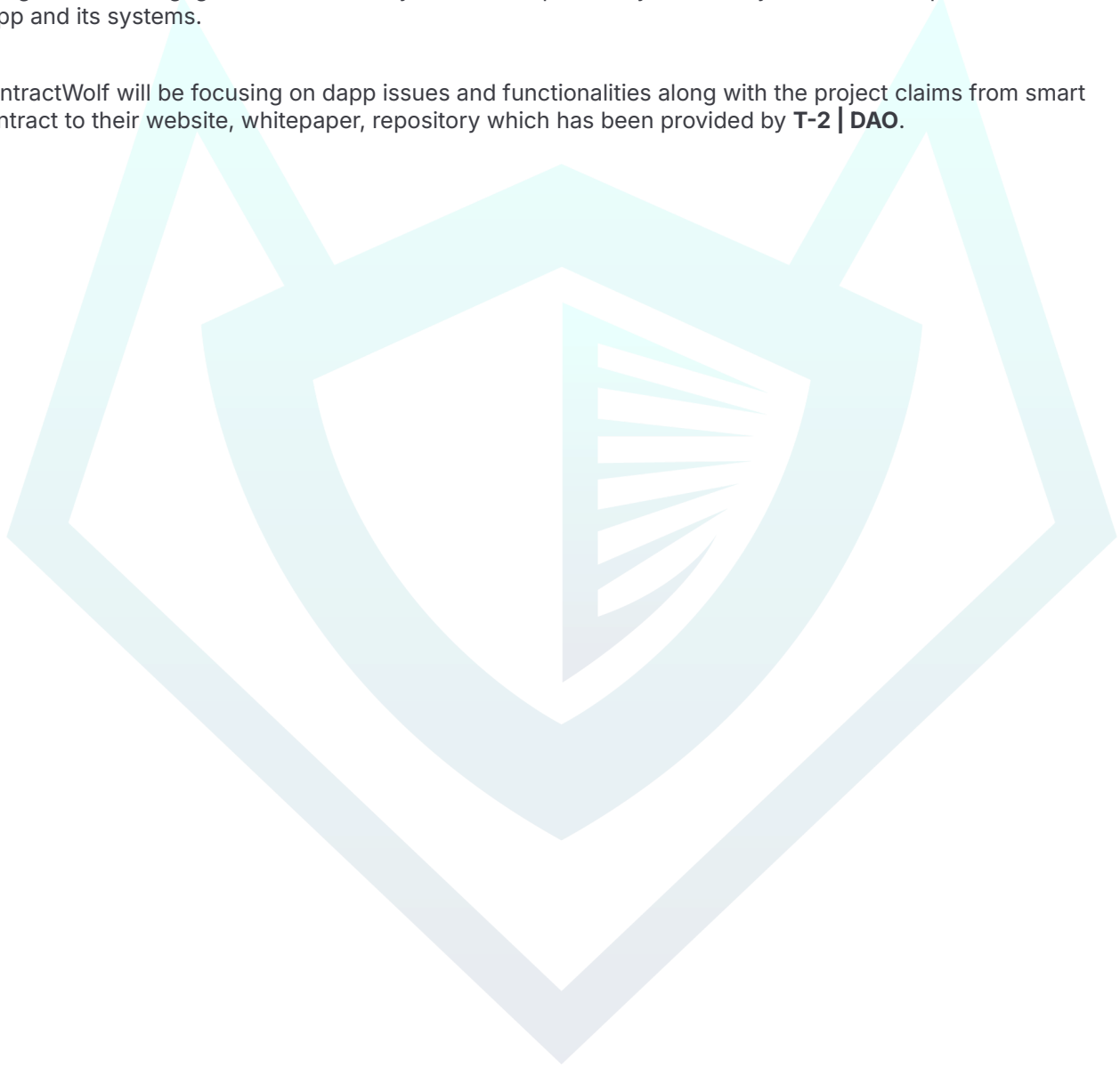
Each company or project should be liable to its security flaws and functionalities.

SCOPE OF WORK | T-2 | DAO

T-2 | DAO's team has agreed and provided us with the files that need to be tested. The scope of audit is the main dapp.

The goal of this engagement is to identify if there is a possibility of security flaws in the implementation of dapp and its systems.

ContractWolf will be focusing on dapp issues and functionalities along with the project claims from smart contract to their website, whitepaper, repository which has been provided by **T-2 | DAO**.



AUDITING APPROACH | T-2 | DAO

Every line of code along with its functionalities will undergo manual review to check for security issues, quality of logic and dapp scope of inheritance. The manual review will be done by our team that will document any issues that they discovered.

METHODOLOGY

The auditing process follows a routine series of steps :

1. Code review that includes the following :
 - Review of the specifications, sources and instructions provided to ContractWolf to make sure we understand the size, scope and functionality of the DAPP.
 - Manual review of code. Our team will have a process of reading the code line-by-line with the intention of identifying potential vulnerabilities, underlying and hidden security flaws.
2. Testing and automated analysis that includes :
 - Testing the DAPP's function with common test cases and scenarios to ensure that it returns the expected results.
3. Best practices and ethical review. The team will review the dapp with the aim to improve efficiency, effectiveness, clarifications, maintainability, security and control within the dapp.
4. Recommendations to help the project take steps to eliminate or minimize threats and secure the dapp.

TOKEN DETAILS

T-2 | DAO



将通过AI构建区块链系统与真实世界的底层链接, 让RWA (真实资产)+AI智能算法+去中心化治理切实落地

Token Name	Symbol	Decimal	Total Supply	Chain
0	0	0	-	Ethereum

SOURCE

Source

Sent Via local-files

FINDINGS | T-2 | DAO



Total Findings

Critical

Major

Medium

Minor

Informational

Resolved

This report has been prepared to state the issues and vulnerabilities for T-2 | DAO Dapp through this audit. The goal of this report findings is to identify specifically and fix any underlying issues and errors

ID	Title	Severity	Status
DCW-018	Overflow or Precision Loss	Major	● Acknowledged
DCW-018	Overflow or Precision Loss	Major	● Acknowledged
DCW-012	Error Handling	Major	● Resolved
DCW-014	Lack of Input Validation	Minor	● Acknowledged
DCW-013	Memory Leak	Minor	● Acknowledged
DCW-017	Request Limit	Minor	● Acknowledged

PENETRATION ATTACKS | T-2 | DAO

Dapp Weakness Classification and Test Cases

ID	Description	Status
DCW-001	Malware Scan	● Passed
DCW-002	Phishing	● Passed
DCW-003	Missing HTTP Headers	● Passed
DCW-004	Valid SSL Certificate	● Passed
DCW-005	Firewalls(Drop & Deny)	● Passed
DCW-006	Potential SQL Injection	● Passed
DCW-007	Framework Version	● Passed
DCW-008	Gas Griefing	● Passed
DCW-009	Address Approval	● Passed
DCW-010	Address Draining	● Passed
DCW-011	Insecure API Usage	● Passed
DCW-012	Error Handling	● Resolved
DCW-013	Memory Leak	● Acknowledged
DCW-014	Lack of Input Validation	● Acknowledged
DCW-015	Potential Backdoor	● Passed
DCW-016	Sensitive Data Exposure	● Passed
DCW-017	Request Limit	● Acknowledged
DCW-018	Overflow or Precision Loss	● Acknowledged
DCW-019	Unintended Behavior	● Passed

FIXES & RECOMMENDATION

DCW-012 | Error Handling

src/utils/request.ts

Axios error interceptors are expected to `return Promise.reject(error)`. This one instead *resolves* with `error.response.data` (or the raw error object for network failures). Because this is the single request client imported by every page (`transfer.vue`, `withdraw.vue`, `trade.vue`, etc.), every failed API call in the app is silently treated as a successful response.

Recommendation:

Return `Promise.reject()` in the error branch instead of resolving, using a normalized error shape so the app still has something predictable to read `.msg` off of. Add `.catch()` at call sites that need to react to a failure.

```
(error) => {
  if (error.config?.showLoadingUI)
    stopLoading()

  // fix the error handling: build a normalized error
  const normalized = error.response
    ? error.response.data
    : { flag: false, msg: i18n.global.t('request.networkError') }

  // and REJECT instead of resolving, so failures are distinguishable from success
  return Promise.reject(normalized)
},
```

```
// example caller update -- transfer.vue
request.post('/opt/transfer', state.form)
  .then((res) => {
    if (res.flag) {
      showSuccessToast(t('success'))
      state.amount = 0
      getAssets()
    }
    else {
      showToast(res.msg)
    }
  })
// fix the error handling: catch is now reachable since the interceptor rejects on failure
.catch((err) => {
  showToast(err.msg || t('request.networkError'))
})
```

DCW-011 | Lack of Input Validation

src/pages/transfer.vue

The transfer destination address field has no format validation before submission. `withdraw.vue` in the same codebase already validates addresses with `isAddress()` from `viem`; `transfer.vue` has no equivalent check, so a malformed address is only caught server-side, if at all.

```
import request from '~/utils/request'

function transfer() {
  if (!Number(state.form.amount)) {
    showToast(t('89'))
    return
  }

  if (Number(state.assets.amount) - Number(state.assets.transferFreeze) <
    Number(state.form.amount)) {
    showToast(t('214'))
    return
  }

  request.post('/opt/transfer', state.form).then((res) => {
    //
  })
}
```

Recommendation:

`isAddress` from `viem` (already used in `withdraw.vue`) and validate `state.form.address` before the amount checks.

```
import { isAddress } from 'viem'
import request from '~/utils/request'

function transfer() {
  // fix the input validation: reject a missing or malformed destination address
  if (!state.form.address || !isAddress(state.form.address)) {
    showToast(t('z99'))
    return
  }

  if (!Number(state.form.amount)) {
    showToast(t('89'))
    return
  }

  if (Number(state.assets.amount) - Number(state.assets.transferFreeze) <
    Number(state.form.amount)) {
```

```
    showToast(t('214'))  
    return  
  }  
  
  request.post('/opt/transfer', state.form).then((res) => {  
    //  
  })  
}
```



DCW-018 | Overflow or Precision Loss

src/pages/transfer.vue, withdraw.vue

Balance and "max amount" calculations use native `Number()` subtraction instead of the project's own decimal-safe `math()` helper (`/utils/mathUtils`, built on `decimal.js`), risking rounding error on the exact values that gate a money-movement action.

Recommendation:

Replace the `Number()` subtraction with `math(a).subtract(b).valueOf()` in both files.

```
// transfer.vue
import { math } from '~/utils/mathUtils'
import request from '~/utils/request'

function all() {
  // fix the decimal handling: Decimal-safe subtraction instead of float Number() math
  const amount = math(state.assets.amount).subtract(state.assets.transferFreeze).valueOf()
  state.form.amount = amount <= 0 ? 0 : amount
}

function transfer() {
  //
  // fix the decimal handling: same Decimal-safe subtraction for the balance guard
  const available = math(state.assets.amount).subtract(state.assets.transferFreeze).valueOf()
  if (available < Number(state.form.amount)) {
    showToast(t('214'))
    return
  }
  //
}
```

```
// withdraw.vue
import { math } from '~/utils/mathUtils'

function all() {
  if (state.selectedCoin.coin === 'USDC') {
    // fix the decimal handling: Decimal-safe subtraction instead of float Number() math
    const amount = math(state.assets.amount).subtract(state.assets.transferFreeze).valueOf()
    state.amount = amount <= 0 ? 0 : amount
  }
  //
}
```

DCW-018 | Overflow or Precision Loss

src/utils/solanaUtils.ts

`sendSol()`, `transferUSDCFromWallet()`, and `transferToken()` convert human-readable amounts to lamports/token base units using raw floating-point multiplication (`* 1e9`, `* 1e6`, `* 10 ** scale`), which can yield a non-integer result for ordinary decimal inputs — fed directly into on-chain transfer instructions that require an exact integer. `erc20Utils.ts`'s `erc20Transfer()` already solves the same problem correctly with a string/BigInt conversion a few files away.

```
// sendSol()
lamports: amountSol * 1e9,

// transferUSDCFromWallet()
amount * 1e6,

// transferToken()
amount * 10 ** scale,
```

Recommendation:

Add one shared `toBaseUnits()` helper using the same string-split → `BigInt` approach as `erc20Utils.ts`, and route all three call sites through it.

```
// fix the decimal handling: exact integer base-unit conversion, no floating point math
/**
 * Converts a human-readable decimal amount to an exact integer number of
 * base units (lamports / token smallest-unit) without floating point math.
 */
function toBaseUnits(amount: number | string, decimals: number): bigint {
  const amountStr = String(amount)
  const [whole = '0', frac = ''] = amountStr.split('.')
  const paddedFrac = (frac + '0'.repeat(decimals)).slice(0, decimals)
  return BigInt(whole + paddedFrac)
}
```

```
// sendSol()
const transaction = new Transaction().add(
  SystemProgram.transfer({
    fromPubkey,
    toPubkey: new PublicKey(toAddress),
    // fix the decimal handling: was `amountSol * 1e9`
    lamports: toBaseUnits(amountSol, 9),
  }),
)
```

```
// transferUSDCFromWallet()
const transferIx = createTransferInstruction(
  senderATA,
  recipientATA,
  fromPublicKey,
  // fix the decimal handling: was `amount * 1e6`
  toBaseUnits(amount, 6),
  [],
  TOKEN_PROGRAM_ID,
)

// transferToken()
transaction.add(
  createTransferInstruction(
    senderATA,
    recipientATA,
    fromPublicKey,
    // fix the decimal handling: was `amount * 10 ** scale`
    toBaseUnits(amount, scale),
    [],
    TOKEN_PROGRAM_ID,
  ),
)
```

DCW-013 | Memory Leak

src/pages/trade.vue

The NFT-trade countdown timer is never cleared when the component unmounts. The cleanup call is wired to a second `onMounted` hook instead of `onUnmounted`, it runs immediately at mount (before the timer even starts) rather than when the user navigates away, so the `setInterval` keeps ticking indefinitely after the page is left.

Recommendation:

Move `stopCountdownJob()` into an `onUnmounted` hook, matching the correct pattern already used in `voting-list.vue`.

```
onMounted(() => {
  getList()
})

// fix the memory leak: clear the countdown interval on unmount, not on mount
onUnmounted(() => {
  stopCountdownJob()
})
```


DCW-017 | Request Limit

src/pages/transfer.vue

The transfer, withdraw, and buy-NFT submit buttons have no disabled/loading guard while their request is in flight, so rapid double-clicking can fire duplicate submissions before client-side state has refreshed.

```
function transfer() {
  if (!Number(state.form.amount)) {
    showToast(t('89'))
    return
  }
  //
  request.post('/opt/transfer', state.form).then((res) => {
    if (res.flag) {
      showSuccessToast(t('success'))
      state.amount = 0
      getAssets()
    }
    else {
      showToast(res.msg)
    }
  })
}

//button
<button class="mt-70 w-full rounded-8 py-12 text-white font-500" @click="transfer"> {{
  $t('submit') }} </button>
```

Recommendation:

Add an in-flight `state.submitting` flag, guard the function's entry point with it, set/clear it around the request, and bind it to the button's `:disabled`. Apply the same pattern to `withdraw()` and `buyNft()`.

```
const state = reactive({
  //
  submitting: false, // fix the request limit: tracks whether a submit is already in flight
})

function transfer() {
  // fix the request limit: block re-entrant submits while one is already in flight
  if (state.submitting)
    return

  if (!Number(state.form.amount)) {
    showToast(t('89'))
    return
  }
```

```
}  
//  
  
// fix the request limit: lock before firing the request  
state.submitting = true  
request.post('/opt/transfer', state.form)  
  .then((res) => {  
    if (res.flag) {  
      showSuccessToast(t('success'))  
      state.amount = 0  
      getAssets()  
    }  
    else {  
      showToast(res.msg)  
    }  
  })  
// fix the request limit: always release the lock, success or failure  
  .finally(() => {  
    state.submitting = false  
  })  
}  
  
// button  
<button class="mt-70 w-full rounded-8 py-12 text-white font-500" :disabled="state.submitting"  
:style="state.submitting ? 'opacity: 0.6;' : 'background: linear-gradient(270deg, #45E1EF 43%,  
#7184F7 100%);'" @click="transfer" > {{ $t('submit') }} </button>
```

AUDIT COMMENTS | T-2 | DAO

Dapp audit comment for a non-technical perspective

- Project has been marked as SAFE to be interacted with by any SVM wallets (07-26-2026)
- DAPP has no backdoors
- DAPP cannot drain wallets via approval





CONTRACTWOLF

Blockchain Security - Smart Contract Audits